

题目 1

- 用户进程的地址空间主要包括代码段、数据段 (Data+BSS)、堆段、栈段、地址空间中的部分用户态不可访问和未使用空间等
 - 未使用空间主要用于**将各个段的起始地址与页面对齐**，避免同一个段的跨页存储
 - 在 NexOS 中，在栈的低地址方向放置一个 Guard Page 页面，以**监控并防止栈越界**
 - 在 NexOS 中，地址空间的最高逻辑地址处保存 **TRAMPOLINE** 和 **TRAPFRAME**，分别用于存储用户态和内核态切换过程中执行的 trampoline 代码、用户寄存器和内核返回信息
 - trampoline 代码所在的页在任意用户页表和内核页表中都映射到同一个页框，通过执行 trampoline 代码实现用户态和内核态相互切换过程中页表和寄存器的转存、读取过程
 - sbrk()** 主要影响**堆段**
- 第一次 **sbrk(6000)** 的返回值是 **0x5000=20480**，第二次 **sbrk(-3000)** 的返回值是 **26480=0x6770**
 - 第一次调用后，新的 **p->sz=0x6770**；第二次调用后，新的 **p->sz=0x5bb8**
- 第一次调用 **sbrk()** 后，**p->sz** 更新为 **0x6770**，因此需要新分配一个用户页，其起始地址为 **0x6000**
 - 第二次调用 **sbrk()** 后，**p->sz** 更新为 **0x5bb8**，因此需要释放一个用户页，其起始地址为 **0x6000**

题目 2

- malloc(40)** 的 **needed** 计算
 - $n + \text{HEADER} = 0x28 + 0x10 = 0x38$
 - 由于对齐标准为 **ALIGN = 0x10**，**needed** 大小向上对齐为 **0x40**
 - 执行 **malloc(40)** 时，发现第一个空闲块 **B** 满足分配条件，因此 **malloc(40)** 使用该块，并返回 **0x1080+0x10=0x1090**
 - malloc(40)** 分配空间后，**B** 块的剩余空间大小 **0x60-0x40=0x20**，因此发生空闲块分裂，产生新块 **0x10c0(0x20)**
- free(p2)** 从 **0x10e0** 开始释放内存空间，**C** 块变为空闲状态，并发现地址上前驱块和后继块都为空闲状态，因此发生合并产生块 **0x10c0(0x120)**，最终的空闲链表为 **0x10c0(0x120)**

题目 3

- Segmentation Fault 指用户进程**尝试以错误的方式访问其地址空间**时出现的错误
 - 包括尝试修改代码段、尝试访问未分配或用户态不可访问的内存区域
- TLB 缓存**近期访问的页面到对应页框的映射关系**，以提高查找页表过程的效率
 - 在多级页表机制下，TLB 对于查找页表过程的性能提升更显著
 - 如果访问的页面已经位于 TLB 中，CPU 可以在一个时钟周期内得到物理内存的访问地址；否则，CPU 先访问物理内存中的进程页表得到访问页面映射的物理页框，再将这个映射关系保存到 TLB 中
- Page Fault：进程尝试访问一个页面，但该页面**尚未映射到物理页框**（包括实际分配的空间在外存中的情形），或**不支持当前访问方式**
(例如对只读的页面尝试写入，或在用户态下尝试访问用户态不可见的页面)
- Demand Paging 是一种虚拟内存管理技术，用于提高系统的进程并行能力
 - 在一个进程开始运行时，只在进程页表中划分出每个段的逻辑地址，但**不将各个页面实际映射到物理页框**
 - 在进程第一次尝试访问某个页面时，触发 Page Fault，此时操作系统将对应的数据从外存加载到内存，建立页面与对应的物理页框的映射关系

- 在实际访问页面时按需分配内存，有利于缩短进程启动时间，提高物理内存实际利用率（而不是分配给在进程运行过程中始终不访问的页面）

5. Thrashing 指分页系统运行时间比程序实际执行时间更长的现象

- 如果系统中并行运行的进程数量过多或部分进程内存访问的时空局部性不强，会导致所有活跃进程的总工作集大小超过物理页框数量
- 物理页框数量不足以满足所有活跃进程的页面访问需求时，在某个进程运行过程中就可能频繁出现 Page Fault 并执行换页操作的问题，导致**分页系统占用大量程序运行时**

题目 4

1. 一次 Page Fault 的平均处理时间为

$$(6 \times (1 - 40\%) + 18 \times 40\%) \text{ ms} = 10.8 \text{ ms}$$

2. TLB 不命中时，访问时间主要由以下过程构成：

- 访问 TLB
- 3 级页表查表时间，即 3 次访问内存
- 如果发生 Page Fault，则处理缺页异常
- 访问内存读取页面数据

因此 TLB 不命中时的平均访问时间为

$$(10 + 3 \times 80 + 0.5\% \times 10.8 \times 10^6 + 80) \text{ ns} = 54330 \text{ ns}$$

3. TLB 命中时，访问时间主要由以下过程构成：

- 访问 TLB
- 访问内存读取页面数据

因此 TLB 命中时的平均访问时间为

$$(10 + 80) \text{ ns} = 90 \text{ ns}$$

从而系统的有效访问时间

$$\text{EAT} = (90 \times 96\% + 54330 \times (1 - 96\%)) \text{ ns} = 2259.6 \text{ ns}$$

4. 若要求系统有效访问时间不超过 150 ns，设 TLB 不命中时的最大可接受的 Page Fault Rate 为 x ，则由

$$\text{EAT} = ((10 + 3 \times 80 + x \times 10.8 \times 10^6 + 80) \times 4\% + 90 \times 96\%) \text{ ns} = 150 \text{ ns}$$

解得 $x = 0.012\%$ 。此时，系统总 Page Fault Rate 为

$$0.012\% \times 4\% = 4.67 \times 10^{-6}$$

题目 5

采用 LRU 替换算法，总共发生 **17 次** Page Fault，具体过程如下：

Visit Idx	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
--------------	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----

Frame 0	7	7	7	1	1	1	3	3	3	7	7	7	7	5	5	5	2	2	2	1
Frame 1		2	2	2	2	2	2	4	4	4	4	1	1	1	4	4	4	3	3	3
Frame 2			3	3	3	5	5	5	6	6	6	6	0	0	0	6	6	6	0	0
Rep. Target Page	7	7	7	2	3	1	2	5	3	4	4	6	7	1	0	5	4	6	2	3

采用 FIFO 替换算法，总共发生 **17 次** Page Fault，具体过程如下：

Visit Idx	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Frame 0	7	7	7	1	1	1	1	1	6	6	6	6	0	0	0	6	6	6	0	0
Frame 1		2	2	2	2	5	2	2	2	7	7	7	7	5	5	5	2	2	2	1
Frame 2			3	3	3	3	3	4	4	4	4	1	1	1	4	4	4	3	3	3
Rep. Target Page	7	7	7	2	2	3	3	1	2	4	4	6	7	1	0	5	4	6	2	3

采用最优（Optimal）替换算法，总共发生 **13 次** Page Fault，具体过程如下：

Visit Idx	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Frame 0	7	7	7	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Frame 1		2	2	2	2	5	5	5	5	5	5	5	5	5	4	6	2	3	3	3
Frame 2			3	3	3	3	3	4	6	7	7	7	0	0	0	0	0	0	0	1
Rep. Target Page	7	7	7	1	2	5	3	4	6	5	7	7	1	5	4	6	2	3	3	3

题目 6

- 虚拟地址 `0x0000002012345678` 对应的 `Offset=0x678`, `PX(0)=0x145`, `PX(1)=0x91`, `PX(2)=0x80`

2.
 - 索引为 `0x80` 的 root page table、索引为 `0x91` 的 level-1 page table、索引为 `0x145` 的 level-0 page table 均存在
 - 该叶子 PTE 映射到一个物理页，且权限为 `PTE_V | PTE_R | PTE_W | PTE_U`
 - 因此虚拟地址 `0x0000002012345678` 可通过 `walkaddr()` 成功翻译，`walkaddr()` 返回值为 `0x0000000081234000`，从而最终物理地址是 `0x0000000081234678`
3.
 - 虚拟地址 `0x0000002012346abc` 对应的 `Offset=0xabc`，`PX(0)=0x146`，`PX(1)=0x91`，`PX(2)=0x80`
 - 索引为 `0x80` 的 root page table、索引为 `0x91` 的 level-1 page table、索引为 `0x146` 的 level-0 page table 均存在
 - 该叶子 PTE 映射到的物理页不具有 `PTE_W` 权限位，因此在用户态对该地址执行写操作时，被 NexOS 判定为非法访问，并**杀死进程**
 - 与尝试写入只读非 COW 页相比，NexOS 在处理 COW 页的写 Fault 时，会解除该页的 COW 状态，在必要时**复制一份物理页用于当前进程写入**，而不是直接杀死进程